

NAME : Mahalakshmi K

ROLLNO : CH.SC.U4CSE24160

WEEK – 5

1) Construct an AVL tree with these numbers:

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

(i) Print the tree showing each level.

(ii) Check that all nodes are balanced

CODE:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node *left, *right;
6      int height;
7  };
8  int max(int a, int b) {
9      return (a > b) ? a : b;
10 }
11 int height(struct Node *n) {
12     if (n == NULL)
13         return 0;
14     return n->height;
15 }
16 struct Node* newNode(int data) {
17     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
18     node->data = data;
19     node->left = node->right = NULL;
20     node->height = 1;
21     return node;
22 }
23 struct Node* rightRotate(struct Node* y) {
24     struct Node* x = y->left;
25     struct Node* T2 = x->right;
26     x->right = y;
27     y->left = T2;
28     y->height = max(height(y->left), height(y->right)) + 1;
29     x->height = max(height(x->left), height(x->right)) + 1;
30     return x;
31 }
32 struct Node* leftRotate(struct Node* x) {
33     struct Node* y = x->right;
34     struct Node* T2 = y->left;
35     y->left = x;
36     x->right = T2;
37     x->height = max(height(x->left), height(x->right)) + 1;
38     y->height = max(height(y->left), height(y->right)) + 1;
39     return y;
40 }
41 int getBalance(struct Node* n) {
42     if (n == NULL)
43         return 0;
44     return height(n->left) - height(n->right);

```

```

45 }
46 struct Node* avlInsert(struct Node* node, int data) {
47     if (node == NULL)
48         return newNode(data);
49     if (data < node->data)
50         node->left = avlInsert(node->left, data);
51     else if (data > node->data)
52         node->right = avlInsert(node->right, data);
53     else
54         return node;
55     node->height = 1 + max(height(node->left), height(node->right));
56     int balance = getBalance(node);
57     // Left Left Case
58     if (balance > 1 && data < node->left->data)
59         return rightRotate(node);
60     // Right Right Case
61     if (balance < -1 && data > node->right->data)
62         return leftRotate(node);
63     // Left Right Case
64     if (balance > 1 && data > node->left->data) {
65         node->left = leftRotate(node->left);
66         return rightRotate(node);
67     }
68     // Right Left Case
69     if (balance < -1 && data < node->right->data) {
70         node->right = rightRotate(node->right);
71         return leftRotate(node);
72     }
73     return node;
74 }
75 void levelOrder(struct Node* root) {
76     if (root == NULL) {
77         printf("Tree is empty\n");
78         return;
79     }
80     struct Node* queue[100];
81     int front = 0, rear = 0;
82     int currentLevel = 0;
83     queue[rear++] = root;
84     printf("\nLevel Order Traversal:\n");
85     while (front < rear) {
86         int levelSize = rear - front;

```

```

87         printf("Level %d: ", currentLevel++);
88         for (int i = 0; i < levelSize; i++) {
89             struct Node* current = queue[front++];
90             printf("%d ", current->data);
91             if (current->left != NULL)
92                 queue[rear++] = current->left;
93             if (current->right != NULL)
94                 queue[rear++] = current->right;
95         }
96         printf("\n");
97     }
98 }
99 void printTree(struct Node* root, int space) {
100     if (root == NULL)
101         return;
102     space += 10;
103     printTree(root->right, space);
104     printf("\n");
105     for (int i = 10; i < space; i++)
106         printf(" ");
107     printf("%d\n", root->data);
108     printTree(root->left, space);
109 }
110 int main() {
111     struct Node* root = NULL;
112     int values[] = {157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117};
113     int n = sizeof(values)/sizeof(values[0]);
114     printf("BUILDING AVL TREE\n");
115     printf("Inserting values: ");
116     for (int i = 0; i < n; i++) {
117         printf("%d ", values[i]);
118         root = avlInsert(root, values[i]);
119     }
120     printf("\n");
121     levelOrder(root);
122     if (getBalance(root) >= -1 && getBalance(root) <= 1)
123         printf("Tree is AVL balanced\n");
124     else
125         printf("Tree is NOT balanced\n");
126     return 0;
127 }

```

Output:

```

BUILDING AVL TREE
Inserting values: 157 110 147 122 149 151 111 141 112 123 133 117

Level Order Traversal:
Level 0: 122
Level 1: 111 147
Level 2: 110 112 133 151
Level 3: 117 123 141 149 157
Tree is AVL balanced

-----
Process exited after 0.07536 seconds with return value 0
Press any key to continue . . .

```

2) Construct a Red-Black tree with the numbers:

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

Print the tree showing R (red) or B (black) for each node. Check that:

(i) Root is black

(ii) No red node has red children

CODE:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #define RED 1
4  #define BLACK 0
5  struct Node {
6      int data;
7      int color;
8      struct Node *left, *right, *parent;
9  };
10 struct Node *root = NULL;
11 struct Node* createNode(int data) {
12     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
13     node->data = data;
14     node->color = RED;
15     node->left = node->right = node->parent = NULL;
16     return node;
17 }
18 void leftRotate(struct Node *x) {
19     struct Node *y = x->right;
20     x->right = y->left;
21
22     if (y->left != NULL)
23         y->left->parent = x;
24
25     y->parent = x->parent;
26
27     if (x->parent == NULL)
28         root = y;
29     else if (x == x->parent->left)
30         x->parent->left = y;
31     else
32         x->parent->right = y;
33
34     y->left = x;
35     x->parent = y;
36 }
37 void rightRotate(struct Node *y) {
38     struct Node *x = y->left;
39     y->left = x->right;
40
41     if (x->right != NULL)
42         x->right->parent = y;
43
44     x->parent = y->parent;

```

```

45
46     if (y->parent == NULL)
47         root = x;
48     else if (y == y->parent->left)
49         y->parent->left = x;
50     else
51         y->parent->right = x;
52
53     x->right = y;
54     y->parent = x;
55 }
56 void fixInsert(struct Node *z) {
57     while (z != root && z->parent->color == RED) {
58
59         if (z->parent == z->parent->parent->left) {
60             struct Node *y = z->parent->parent->right; // Uncle
61
62             // Case 1: Uncle is RED
63             if (y != NULL && y->color == RED) {
64                 z->parent->color = BLACK;
65                 y->color = BLACK;
66                 z->parent->parent->color = RED;
67                 z = z->parent->parent;
68             }
69             // Case 2 & 3
70             else {
71                 if (z == z->parent->right) {
72                     z = z->parent;
73                     leftRotate(z);
74                 }
75                 z->parent->color = BLACK;
76                 z->parent->parent->color = RED;
77                 rightRotate(z->parent->parent);
78             }
79         }
80         else {
81             struct Node *y = z->parent->parent->left;
82
83             if (y != NULL && y->color == RED) {
84                 z->parent->color = BLACK;
85                 y->color = BLACK;
86                 z->parent->parent->color = RED;

```

```

87         z = z->parent->parent;
88     }
89     else {
90         if (z == z->parent->left) {
91             z = z->parent;
92             rightRotate(z);
93         }
94         z->parent->color = BLACK;
95         z->parent->parent->color = RED;
96         leftRotate(z->parent->parent);
97     }
98 }
99 }
100 root->color = BLACK;
101 }
102 void insert(int data) {
103     struct Node *z = createNode(data);
104     struct Node *y = NULL;
105     struct Node *x = root;
106
107     while (x != NULL) {
108         y = x;
109         if (z->data < x->data)
110             x = x->left;
111         else
112             x = x->right;
113     }
114
115     z->parent = y;
116
117     if (y == NULL)
118         root = z;
119     else if (z->data < y->data)
120         y->left = z;
121     else
122         y->right = z;
123
124     fixInsert(z);
125 }
126 int height(struct Node* node) {
127     if (node == NULL) return 0;
128     int leftHeight = height(node->left);

```

```

129     int rightHeight = height(node->right);
130     return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;
131 }
132 void printLevel(struct Node* node, int level, int currentLevel) {
133     if (node == NULL) {
134         if (currentLevel == level) printf(" ");
135         return;
136     }
137     if (currentLevel == level) {
138         printf("%d(%s) ", node->data, node->color == RED ? "R" : "B");
139     } else if (currentLevel < level) {
140         printLevel(node->left, level, currentLevel + 1);
141         printLevel(node->right, level, currentLevel + 1);
142     }
143 }
144 void levelOrder() {
145     int h = height(root);
146     printf("\n\nLevel Order Traversal:\n");
147     for (int i = 0; i < h; i++) {
148         printf("Level %d: ", i);
149         printLevel(root, i, 0);
150         printf("\n");
151     }
152 }
153 void inorder(struct Node *node) {
154     if (node != NULL) {
155         inorder(node->left);
156         printf("%d(%s) ", node->data,
157             node->color == RED ? "R" : "B");
158         inorder(node->right);
159     }
160 }
161 int main() {
162     printf("Red-Black Tree Implementation\n");
163     printf("===== \n\n");
164
165     int keys[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
166     int n = sizeof(keys) / sizeof(keys[0]);
167
168     printf("Inserting values in order:\n");
169     for (int i = 0; i < n; i++) {
170         printf("%d ", keys[i]);
171         insert(keys[i]);
172     }
173
174     printf("\n\nInorder Traversal:\n");
175     inorder(root);
176     levelOrder();
177     return 0;
178 }

```

OUTPUT:

```
Red-Black Tree Implementation
=====

Inserting values in order:
157 110 147 122 111 149 151 141 123 112 117 133

Inorder Traversal:
110(B) 111(R) 112(R) 117(B) 122(R) 123(B) 133(R) 141(B) 147(R) 149(R) 151(B) 157(R)

Level Order Traversal:
Level 0: 123(B)
Level 1: 111(R) 147(R)
Level 2: 110(B) 117(B) 141(B) 151(B)
Level 3:      112(R) 122(R) 133(R)      149(R) 157(R)

-----
Process exited after 0.09999 seconds with return value 0
Press any key to continue . . . |
```